

Introducción al Procesamiento de Lenguaje Natural (NLP) con NLTK

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: fprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



12 de febrero de 2026

Procesamiento de Lenguaje Natural (NLP)^a

- ▶ El NLP es una disciplina de la inteligencia artificial que se enfoca en **hacer que los programas informáticos comprendan y utilicen el lenguaje humano de forma efectiva.**
- ▶ NLP permite que las computadoras **interactúen con texto o habla humana** de una manera significativa.
- ▶ Se aplica en tareas como traducción automática, análisis de sentimientos, reconocimiento de voz, etc.

^aTomado de: [Natural Language Processing With Python's NLTK Package](#).

NLTK: Natural Language Toolkit

- ▶ **NLTK** (Natural Language Toolkit) es una biblioteca de Python diseñada específicamente para trabajar con texto en tareas de NLP.
- ▶ Proporciona herramientas para:
 - ▶ Tokenización
 - ▶ Eliminación de stopwords
 - ▶ Lematización y stemming
 - ▶ Análisis sintáctico y gramatical

Datos No Estructurados

- ▶ Gran parte de los datos disponibles hoy en día son **no estructurados** y contienen texto legible por humanos.
- ▶ Ejemplos: artículos, comentarios, publicaciones en redes sociales, correos electrónicos.
- ▶ Para analizarlos de manera computacional, es necesario realizar una **preparación o preprocesamiento**.

Preprocesamiento del Texto

- ▶ El **preprocesamiento convierte el texto crudo en un formato más estructurado y útil para el análisis.**
- ▶ Algunas tareas comunes incluyen:
 - ▶ Conversión a minúsculas
 - ▶ Eliminación de puntuación y caracteres especiales
 - ▶ Tokenización (dividir en palabras o frases)
 - ▶ Eliminación de palabras vacías (stopwords)
 - ▶ Lematización o stemming:
 - ▶ Lematización: proceso que reduce una palabra a su forma base correcta (lema) según su significado gramatical. Ej:
"corriendo" → "correr"
 - ▶ Stemming: proceso que recorta una palabra hasta su raíz eliminando sufijos.

Tokenización

- ▶ La **tokenización** es el proceso de **dividir un texto en unidades lingüísticas más pequeñas**, llamadas *tokens*.
- ▶ Estos tokens pueden ser palabras, frases o incluso oraciones completas.
- ▶ Es el **primer paso para transformar datos no estructurados en datos estructurados**, más fáciles de analizar.
- ▶ Permite trabajar con fragmentos coherentes y significativos del texto.

Formalización

- ▶ Sea un texto T compuesto por una secuencia de caracteres:

$$T = c_1, c_2, \dots, c_n$$

- ▶ La tokenización define una función:

$$\mathcal{T}(T) = \{t_1, t_2, \dots, t_k\}$$

donde cada t_i representa una palabra u oración, según el criterio de segmentación adoptado.

Tipos de Tokenización

- ▶ **Tokenización por palabra:** divide el texto en palabras individuales.
- ▶ **Tokenización por oración:** divide el texto en oraciones completas.
- ▶ Ambos métodos permiten **distintos niveles de análisis dependiendo del objetivo** del estudio.
- ▶ El **tipo de tokenización determina el nivel de granularidad del análisis** y debe seleccionarse de acuerdo con el objetivo de la aplicación de PLN.

Tokenización por Palabra

- ▶ Las palabras son las **unidades mínimas de significado** que pueden tener sentido por sí solas.
- ▶ Tokenizar por palabra permite identificar términos frecuentes o relevantes.
- ▶ **Ejemplo de uso:**
 - ▶ En un conjunto de anuncios de trabajo, puedes descubrir que la palabra “*Python*” aparece con frecuencia.
 - ▶ Esto podría sugerir una alta demanda de conocimientos en Python.
 - ▶ Sin embargo, se requiere un análisis contextual para confirmar esta hipótesis.

Tokenización por Oración

- ▶ Permite analizar el **contexto completo** en el que aparecen las palabras.
- ▶ Ayuda a detectar relaciones semánticas entre términos.
- ▶ **Ejemplo de reflexión:**
 - ▶ ¿La palabra “Python” aparece junto a términos negativos?
 - ▶ ¿Se usa en un **contexto biológico** (herpetología¹) **en lugar de programación?**

Ejemplo de reflexión

- ▶ Este tipo de análisis ayuda a **interpretar con mayor precisión** el significado de las palabras.

¹Rama de la zoología que se encarga del estudio de los anfibios y los reptiles.

Ejemplo:

Cadena de texto, para análisis, tomada de Dune:

Muad'Dib learned rapidly because his first training was in how to learn. And the first lesson of all was the basic trust that he could learn. It's shocking to find how many people do not believe they can learn, and how many more believe learning to be difficult.

Tokenización por Oración con NLTK

Código en Python:

```
from nltk.tokenize import sent_tokenize

sentences = sent_tokenize(example_string)
for s in sentences:
    print(s)
```

Resultado Esperado:

- ▶ Divide el texto en 3 oraciones completas.
- ▶ Cada oración es una unidad de análisis contextual.

Ver código python.

Tokenización por Palabra con NLTK

Código en Python:

```
from nltk.tokenize import word_tokenize

words = word_tokenize(example_string)
print(words[:15]) # Muestra las primeras 15 palabras
```

Resultado Esperado:

- ▶ Divide el texto en palabras individuales (tokens).
- ▶ Incluye puntuación como tokens separados.

Ver código python.

Observaciones Clave

- ▶ La tokenización por oración permite analizar el contexto y estructura general del texto.
- ▶ La tokenización por palabra es útil para conteo de frecuencias, detección de palabras clave y análisis más fino.
- ▶ Herramientas como `nltk` manejan bien contracciones y puntuación, lo que mejora la calidad del análisis.

Comportamiento de la Tokenización en NLTK

- ▶ NLTK es capaz de **dividir correctamente las contracciones** en palabras separadas, como “It’s” en “It” y “s”.
- ▶ En este caso:
 - ▶ “It’s” se divide en “It” y “s”.
 - ▶ El “s” es tratado como una contracción de “is”.
- ▶ Sin embargo, **palabras como “Muad’Dib” se mantienen intactas**, ya que no son consideradas contracciones.
- ▶ NLTK no descompone “Muad’Dib” en dos palabras, **porque no sigue una regla de contracción conocida**.

Ejemplo de Comportamiento con Contracciones

- ▶ Código en Python para ver el resultado:

```
words = word_tokenize("It's shocking how Muad'Dib  
learned rapidly.")  
print(words)
```

- ▶ Resultado Esperado:

- ▶ `['It', 's', 'shocking', 'how', 'MuadDib', 'learned', 'rapidly', '.']`
- ▶ `"It's"` se tokeniza como `['It', 's']`, mientras que `"Muad'Dib"` se mantiene como una palabra.

Análisis de Composición de Palabras

- ▶ **NLTK es capaz de dividir contracciones comunes debido a su conocimiento de reglas lingüísticas predefinidas.**
- ▶ Sin embargo, para palabras como “Muad'Dib”, que no se ajustan a una regla estándar de contracción, la tokenización respeta la unidad completa de la palabra.
- ▶ Esto muestra cómo NLTK maneja la tokenización de manera inteligente para distintos tipos de texto.

Filtrado de Stop Words.

El filtrado de *stop words* es una etapa del preprocesamiento cuyo objetivo es **eliminar términos de alta frecuencia que aportan escasa información semántica al análisis del texto.**

- ▶ Las **stop words** son palabras muy comunes que suelen eliminarse durante el preprocesamiento del texto.
- ▶ Estas palabras no aportan mucho significado al análisis, por ejemplo: “in”, “is”, “an”.
- ▶ Filtrarlas permite enfocarse en términos más relevantes que aportan información útil.

Formalización

- ▶ Sea el conjunto de tokens obtenidos tras la tokenización:

$$\mathcal{T} = \{t_1, t_2, \dots, t_n\}$$

y sea $\mathcal{S}$ el conjunto de *stop words* predefinidas.

- ▶ El proceso de filtrado se define como:

$$\mathcal{T}' = \{t_i \in \mathcal{T} \mid t_i \notin \mathcal{S}\}$$

donde $\mathcal{T}'$ representa el conjunto de tokens informativos.

Impacto del filtrado de Stop Words en el análisis

- ▶ **Reduce la dimensionalidad** del espacio de características.
- ▶ **Disminuye el ruido** en modelos estadísticos y de aprendizaje automático.
- ▶ **Facilita la identificación de términos discriminativos** relevantes.

Importación de Stop Words en NLTK

- ▶ Código en Python para importar y utilizar stop words:

```
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize
```

```
example_text = "This is a simple example demonstrating  
stop word filtering."
```

```
stop_words = set(stopwords.words('english'))
```

```
words = word_tokenize(example_text)
```

```
filtered_words = [word for word in words if  
word.lower() not in stop_words]
```

```
print(filtered_words)
```

Resultado del Filtrado

- ▶ El texto original: “This is a simple example demonstrating stop word filtering.”
- ▶ Después del filtrado de stop words, podría quedar como:

```
['This', 'simple', 'example', 'demonstrating',  
 'stop', 'word', 'filtering', '.']
```
- ▶ Se eliminaron palabras como “is” y “a”, que no aportan significado relevante.

Ver código python.

Tokenización de una Frase

- ▶ Frase original:

```
worf_quote = ‘‘Sir, I protest. I am not a merry man!’’
```

- ▶ Tokenización por palabra:

```
words_in_quote = word_tokenize(worf_quote)  
print(words_in_quote)
```

- ▶ Resultado:

```
['Sir', ',', 'protest', '.', 'merry', 'man', '!']
```

Definición del Filtro de Stop Words

- ▶ Se usan las **stop words en inglés** de NLTK para eliminar palabras comunes:

```
from nltk.corpus import stopwords  
stop_words = set(stopwords.words('english'))
```

- ▶ También se define una lista vacía para guardar las palabras filtradas:

```
filtered_list = []
```


Filtrado Usando un Bucle For

- ▶ Código:

```
for word in words_in_quote:
    if word.casefold() not in stop_words:
        filtered_list.append(word)
```

- ▶ **Nota:** Se usa `.casefold()` para ignorar mayúsculas y minúsculas, ya que las stop words de NLTK están en minúscula.

Resultado del Filtrado

- ▶ Lista filtrada:
`['Sir', ',', 'protest', '.', 'merry', 'man', '!']`
- ▶ Se eliminaron palabras como “am” y “a”, que son comunes.
- ▶ Pero también se eliminó “not”, lo cual **sí afecta el significado** de la frase original.
- ▶ **Advertencia:** eliminar palabras como “not” puede distorsionar el sentido.
- ▶ En algunos casos, puede ser necesario **personalizar la lista de stop words**, por ejemplo, excluyendo palabras como “not”, “no”, o “never” que tienen carga semántica importante.

Ver código python.

Alternativa con List Comprehension

Código equivalente con list comprehension:

```
filtered_list = [word for word in words_in_quote  
                 if word.casefold() not in stop_words]
```

- ▶ Es una alternativa más compacta y elegante.
- ▶ Define la lista y su contenido en una sola línea.

Palabras como 'I' y 'not': ¿Filtrar o Conservar?

- ▶ Algunas palabras como "I" y "not" pueden parecer demasiado importantes para ser filtradas.
- ▶ Su importancia depende del tipo de análisis que se desea realizar.
- ▶ Estas palabras pueden proporcionar información valiosa sobre:
 - ▶ **Estilo de escritura** ("I")
 - ▶ **Sentimiento o polaridad** ("not")

Palabras de Contenido vs Palabras de Contexto

► Palabras de contenido:

- Aportan significado temático al texto.
- Ejemplos: “protest”, “man”, “happy”.

► Palabras de contexto:

- Informan sobre el estilo del autor.
- Ejemplo: el uso frecuente de “I” puede indicar un estilo introspectivo o narrativo en primera persona.

El Caso Especial de 'not'

- ▶ “not” es técnicamente un adverbio.
- ▶ A pesar de su impacto en el significado de una oración, aparece en la lista de stop words de NLTK.
- ▶ **Filtrar “not” puede eliminar la negación de una frase, alterando completamente su interpretación:**
 - ▶ “I am not happy” → “I am happy” (¡interpretación errónea!)

Personalización de Stop Words en NLTK

- ▶ Se puede editar la lista de stop words para conservar palabras importantes:

```
stop_words = set(stopwords.words('english'))  
stop_words.discard('not')  
stop_words.discard('i')
```

- ▶ Así se puede adaptar la lista al caso de análisis.
- ▶ Es buena práctica revisar la lista de stop words antes de usarla.
- ▶ ['Sir', ',', 'I', 'protest', '.', 'I', 'not', 'merry', 'man', '!']
- ▶ Esto permite mantener el sentido original de la oración.

Remarcas

- ▶ Palabras como “I” y “not” pueden ser esenciales dependiendo del objetivo del análisis.
- ▶ No siempre es conveniente usar la lista de stop words predeterminada.
- ▶ Personalizar la lista te permite un análisis más preciso y controlado.

Ver código python.

Stemming

Es una técnica de preprocesamiento del texto cuyo **propósito es reducir las palabras a su forma raíz o stem**, eliminando sufijos y variaciones morfológicas.

- ▶ Este proceso permite agrupar distintas formas flexionadas de una palabra bajo una misma representación, facilitando el análisis estadístico y reduciendo la complejidad del vocabulario.

Formalización

- ▶ Sea $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ el conjunto de tokens resultante del preprocesamiento previo.
- ▶ El stemming se modela mediante una función:

$$\mathcal{S}(t_i) = \text{stem}(t_i)$$

donde $\mathcal{S}$ transforma cada token en su raíz morfológica.

Ejemplo ilustrativo

- ▶ Las palabras:

“helping”, “helper”

- ▶ son transformadas al mismo *stem*:

“help”

- ▶ lo que permite tratarlas como equivalentes desde el punto de vista del análisis automático.

Algoritmos de stemming

Existen diversos algoritmos implementados en bibliotecas de PLN, entre ellos:

- ▶ **Porter Stemmer:**

Algoritmo basado en reglas secuenciales que elimina sufijos comunes, ofreciendo un buen equilibrio entre simplicidad y precisión.

- ▶ **Lancaster Stemmer:**

Algoritmo más agresivo que produce raíces más cortas, con mayor riesgo de sobre-stemming.

- ▶ **Snowball Stemmer:**

Versión mejorada y extensible del Porter, con soporte para múltiples idiomas.

Aquí se emplea **Porter Stemmer**, debido a su **equilibrio entre efectividad, estabilidad y bajo costo computacional**.

Código para usar el Porter Stemmer de NLTK

```
from nltk.stem import PorterStemmer

# Crear instancia del stemmer
stemmer = PorterStemmer()
```

- ▶ Esta instancia puede aplicarse a palabras individuales con el método `stem()`.
- ▶ Ejemplo:

```
stemmer.stem("helping") # Resultado: 'help'
```

Ejemplo

Texto de entrada:

```
string_for_stemming = ‘‘ The crew of the USS Discovery  
discovered many discoveries. Discovering is what explorers
```

```
# Tokenizar el texto
```

```
words = word_tokenize(string_for_stemming)
```

```
# Aplicar el stemmer
```

```
stemmed_words = [stemmer.stem(word) for word in words]
```

Resultado:

- ▶ Palabras con raíz “discov” obtienen formas distintas tras el stemming.

Ver código python.

Comparación de Resultados del Stemming

Resultados del stemming aplicado:

Palabra original	Versión con stemmer
"Discovery"	"discoveri"
"discovered"	"discov"
"discoveries"	"discoveri"
"Discovering"	"discov"

Observación:

- ▶ El resultado puede parecer inconsistente porque el **Porter Stemmer aplica reglas heurísticas, no basadas en el significado semántico.**
- ▶ **No garantiza obtener la raíz completa o una palabra válida** en inglés, solo una forma reducida útil para agrupación.

Problemas del Stemming:

► Understemming:

Ocurre cuando dos palabras relacionadas no se reducen al mismo stem.

- Ejemplo: “connect” y “connected” → se esperan el mismo stem, pero no ocurre.
- Se considera un **falso negativo**.

► Overstemming:

Ocurre cuando dos palabras no relacionadas se reducen al mismo stem.

- Ejemplo: “universe” y “university” → podrían reducirse a “univers”.
- Se considera un **falso positivo**.

Mejoras al Algoritmo de Porter

- ▶ El algoritmo de **Porter** fue desarrollado en 1979.
- ▶ Está **basado en reglas heurísticas, no en el significado semántico**.
- ▶ Su propósito es agrupar variantes morfológicas de una palabra, no producir palabras completas.
- ▶ **Una versión mejorada es el Snowball Stemmer** (también llamado *Porter2*):
 - ▶ Más preciso.
 - ▶ Disponible también en NLTK.

- ▶ Cuando el stemming produce resultados erróneos o no deseados, es recomendable usar una técnica más precisa: la **lemmatización**.
- ▶ La lematización toma en cuenta el **significado** y la **categoría gramatical** de las palabras.
- ▶ Antes de aplicar lematización, es necesario identificar las **partes del habla** (Part-of-Speech, POS).

Introducción al POS Tagging

El *POS Tagging* (Part-of-Speech Tagging) es el proceso mediante el cual se **asigna una categoría gramatical a cada palabra de un texto, de acuerdo con su función sintáctica dentro de la oración.**

- ▶ Este procedimiento **permite identificar el rol lingüístico de cada término**, como sustantivo, verbo, adjetivo o adverbio, lo cual es esencial para una interpretación estructural del lenguaje natural.

Formalización

- ▶ Sea una secuencia de tokens:

$$\mathcal{T} = (t_1, t_2, \dots, t_n)$$

- ▶ El POS tagging busca determinar la secuencia de etiquetas gramaticales:

$$\mathcal{Y} = (y_1, y_2, \dots, y_n)$$

donde cada y_i pertenece a un conjunto finito de categorías gramaticales $\mathcal{C}$.

Por ejemplo:

- ▶ **NN** (Noun): sustantivo. Ejemplo: dog, city.
- ▶ **VB** (Verb): verbo (forma base). Ejemplo: run, eat.
- ▶ **JJ** (Adjective): adjetivo. Ejemplo: big, fast.

Importancia en PLN

- ▶ **Facilita procesos posteriores** como la lematización y el análisis sintáctico.
- ▶ Mejora la desambiguación léxica al considerar el contexto gramatical.
- ▶ Constituye un paso clave en sistemas de comprensión del lenguaje.
- ▶ **Herramientas:**
Bibliotecas como **NLTK** proporcionan métodos automáticos para realizar POS tagging empleando modelos estadísticos y reglas lingüísticas.

Las 8 Partes del Habla en Inglés

Categoría	Rol	Ejemplos
Noun (sustantivo)	Persona, lugar o cosa	mountain, bagel, Poland
Pronoun (pronombre)	Reemplaza un sustantivo	you, she, we
Adjective (adjetivo)	Describe un sustantivo	efficient, windy, colorful
Verb (verbo)	Acción o estado	learn, is, go
Adverb (adverbio)	Describe un verbo, adjetivo u otro adverbio	efficiently, always, very
Preposition (preposición)	Conecta un sustantivo con otra palabra	from, about, at
Conjunction (conjunción)	Une palabras o frases	so, because, and
Interjection (interjección)	Exclamación	yay, ow, wow

- ▶ Algunos recursos también incluyen a los **artículos** (“a”, “the”) como una categoría separada.
- ▶ En NLTK, estos se etiquetan como **Determiners (DT)**.
- ▶ Las etiquetas POS en NLTK siguen el estándar del corpus **Penn Treebank**, que usa códigos como:
 - ▶ NN: noun, singular
 - ▶ VB: verb, base form
 - ▶ JJ: adjective
 - ▶ RB: adverb
- ▶ Estas etiquetas son más detalladas que las 8 categorías clásicas.

Código para POS Tagging con NLTK

```
from nltk.tokenize import word_tokenize
from nltk import pos_tag

sagan_quote = ‘‘If you wish to make an apple pie
    from scratch, you must first invent the universe.’’

words_in_sagan_quote = word_tokenize(sagan_quote)
tagged_words = pos_tag(words_in_sagan_quote)
print(tagged_words)
```

Ejemplo de Salida:

- ▶ ('If', 'IN'), ('you', 'PRP'), ('wish', 'VBP'), ...
- ▶ Cada tupla contiene una palabra y su etiqueta gramatical.

Ver código python.

Significado de las Etiquetas POS

- ▶ Las etiquetas siguen el estándar del **Penn Treebank**.
- ▶ Ejemplos comunes:
 - ▶ **NN** – sustantivo singular (“pie”, “universe”)
 - ▶ **VB** – verbo en forma base (“make”, “invent”)
 - ▶ **VBP** – verbo presente, forma base no tercera persona (“wish”)
 - ▶ **PRP** – pronombre personal (“you”)
 - ▶ **DT** – determinante (“an”, “the”)
 - ▶ **IN** – preposición o conjunción subordinada (“if”, “from”)
 - ▶ **MD** – verbo modal (“must”)
- ▶ Todas las etiquetas se pueden consultar con:
 - ▶ `nltk.help.upenn_tagset()`

POS Tagging en Texto Sin Sentido

Fragmento del poema Jabberwocky:

```
jabberwocky_excerpt = ‘‘  
'Twas brillig, and the slithy toves did gyre and gimble  
    in the wabe: all mimsy were the borogoves,  
    and the mome raths outgrabe.’’
```

Tokenización y etiquetado:

```
words_in_excerpt = word_tokenize(jabberwocky\_excerpt)  
nltk.pos_tag(words_in_excerpt)
```

Resultados del Etiquetado POS

Ejemplos de etiquetas asignadas:

- ▶ “slithy” → JJ (adjetivo)
- ▶ “toves” → NNS (sustantivo plural)
- ▶ “gyre” → NN (sustantivo)
- ▶ “mimsy” → NNS (sustantivo plural)
- ▶ “mome” → JJ (adjetivo)
- ▶ “outgrabe” → RB (adverbio)

Observaciones:

- ▶ Palabras reales como “and” (CC) y “the” (DT) fueron correctamente etiquetadas.
- ▶ NLTK asignó etiquetas razonables a palabras inventadas usando el contexto gramatical.
- ▶ Muestra cómo el modelo estadístico de NLTK puede inferir funciones lingüísticas incluso en lenguaje no convencional.

Lemmatizing (Lematización)

La *lematización* es una técnica de preprocesamiento del lenguaje natural que, al igual que el *stemming*, **reduce las palabras a su forma base**.

Sin embargo, a diferencia del stemming, **la lematización produce siempre una palabra válida del idioma**, denominada *lema*.

- ▶ Un **lema** corresponde a la forma canónica que representa un conjunto de palabras flexionadas, conocido como *lexema*.

Formalización

- ▶ Sea $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ el conjunto de tokens procesados.
- ▶ La lematización se define mediante una función:

$$\mathcal{L}(t_i, y_i) = \text{lema}(t_i)$$

donde y_i representa la etiqueta gramatical (*POS*) del token, utilizada para resolver ambigüedades lingüísticas.

Ejemplo ilustrativo

- ▶ Sea el lexema:

{blend, blends, blended, blending}

- ▶ La lematización asocia todas estas formas al mismo lema:

blend

- ▶ **Observación:**

El conocimiento del tipo gramatical de cada palabra (verbo, sustantivo, adjetivo, etc.) es fundamental para realizar una lematización correcta, razón por la cual este proceso suele apoyarse en técnicas de *POS tagging*.

Lemmatizing vs Stemming

- ▶ **Stemming:** Produce raíces que no necesariamente son palabras reales. Ej: “discovering” → “discov”
- ▶ **Lemmatizing:** Devuelve el lema correcto. Ej: “discovering” → “discover”

Ventajas del lematizador:

- ▶ Conserva el significado correcto dentro del contexto gramatical.
- ▶ Útil para análisis semántico, recuperación de información y clasificación de texto.

Herramienta en NLTK: WordNetLemmatizer

Ejemplo Práctico de Lemmatizing

- ▶ Importar e inicializar:

```
from nltk.stem import WordNetLemmatizer  
from nltk.tokenize import word_tokenize
```

```
lemmatizer = WordNetLemmatizer()
```

- ▶ Lemmatizar sustantivos plurales:

```
lemmatizer.lemmatize('scarves') # -> 'scarf'  
string = 'The friends of DeSoto love scarves.'  
words = word_tokenize(string)  
lemmatized = [lemmatizer.lemmatize(w) for w in words]
```

- ▶ Resultado:

```
['The', 'friend', 'of', 'DeSoto', 'love', 'scarf', '.']
```


Mejorando la Lematización con POS Tagging

- ▶ **Importancia del parámetro pos:**
- ▶ **Problema:** Algunas palabras tienen múltiples significados según su categoría gramatical.

- ▶ **Ejemplo:**

```
lemmatizer.lemmatize("worst")  
# → 'worst' (asume sustantivo)  
lemmatizer.lemmatize("worst", pos="a")  
# → 'bad' (como adjetivo)
```

- ▶ **Solución:** Etiquetar palabras con su categoría antes de lematizarlas.

Especificar la Categoría Gramatical (POS)

Para obtener un lema más preciso durante la lematización, se puede indicar manualmente la **parte de la oración (POS)** de la palabra.

► Ejemplo:

```
lemmatizer.lemmatize("worst", pos="a")  
# Resultado: 'bad'
```

En este caso, el lematizador devuelve 'bad', que corresponde al **lema correcto** de la palabra 'worst'.

► Comparación con Stemming:

- Técnicas de stemming no pueden reducir 'worst' a 'bad'.
- Esto se debe a que no comparten una raíz textual común.

► Ventaja de la lematización:

- Puede reducir formas **comparativas y superlativas** a su **lema gramatical**.

Recomendación: Etiquetar antes de Lematizar

Dado que muchas palabras pueden tener diferentes significados o funciones gramaticales (por ejemplo, sustantivo o verbo), se recomienda seguir los siguientes pasos:

1. **Tokenizar** el texto.
2. **Etiquetar** cada palabra con su parte de la oración usando `nltk.pos_tag()`.
3. **Convertir** las etiquetas al formato de WordNet.
4. **Lematizar** cada palabra con su etiqueta correspondiente.

Ventaja principal:

- ▶ Este enfoque es especialmente útil para evitar errores con **homógrafos**, es decir, palabras que se escriben igual pero tienen distintos **significados** o **funciones gramaticales**.

Chunking: Identificación de Frases

El *chunking* es una técnica del procesamiento de lenguaje natural orientada a la **identificación de frases o constituyentes superficiales dentro de un texto**, a partir de las etiquetas gramaticales (*POS tags*) previamente asignadas a cada palabra.

- ▶ A diferencia de la tokenización, que segmenta el texto en unidades básicas, el chunking agrupa secuencias de palabras en **unidades gramaticales funcionales**, tales como sintagmas nominales, verbales o preposicionales.

Formalización

- Sea una secuencia de tokens y etiquetas:

$$\mathcal{X} = \{(t_1, y_1), (t_2, y_2), \dots, (t_n, y_n)\}$$

- El chunking define una función:

$$\mathcal{C}(\mathcal{X}) = \{c_1, c_2, \dots, c_m\}$$

donde cada c_j representa una frase gramatical compuesta por uno o más tokens consecutivos.

Ejemplo conceptual

- ▶ Dada una secuencia etiquetada:

(The/DT, quick/JJ, fox/NN)

el chunking identifica el siguiente sintagma nominal:

[The quick fox]_{NP}

- ▶ **Nota:**

Una **frase** corresponde a una palabra o conjunto de palabras que actúan como una unidad estructural y cumplen una función gramatical específica dentro de la oración.

Ejemplos de Sintagmas Nominales

- ▶ Los siguientes ejemplos ilustran cómo se puede construir un **sintagma nominal** agregando elementos alrededor de un sustantivo:
 - ▶ “A planet”
 - ▶ “A tilting planet”
 - ▶ “A swiftly tilting planet”
- ▶ En cada caso, el sustantivo *planet* está modificado por adjetivos o adverbios, pero el conjunto sigue funcionando como una única unidad gramatical.

Funcionamiento del Chunking

- ▶ El **chunking** utiliza las etiquetas gramaticales asignadas en la etapa de etiquetado (POS tagging) para agrupar las palabras en **chunks**, como por ejemplo:
 - ▶ Sintagmas nominales (NP: *Noun Phrases*)
 - ▶ Sintagmas verbales (VP: *Verb Phrases*)
- ▶ Cada chunk se etiqueta con una categoría, como NP o VP, dependiendo del tipo de frase.
- ▶ **Importante:**
 - ▶ Los chunks no se solapan.
 - ▶ Una palabra puede pertenecer solamente a un chunk a la vez.

Preparación para el Chunking

- ▶ Antes de aplicar **chunking**, es necesario tokenizar el texto y etiquetar cada palabra con su categoría gramatical (POS).
- ▶ **Ejemplo:** Cita de *The Lord of the Rings*

```
from nltk.tokenize import word_tokenize
```

```
lotr_quote = 'It's-a-dangerous-business ,  
----- Frodo , -going-out-your-door.'
```

```
words_in_lotr_quote = word_tokenize(lotr_quote)
```

- ▶ El resultado de `word_tokenize` es una lista de palabras separadas:

```
['It', 's', 'a', 'dangerous', 'business', ',',  
'Frodo', ',', 'going', 'out', 'your', 'door', '.']
```

Etiquetado Gramatical (POS Tagging)

- ▶ Una vez tokenizado el texto, se etiquetan las palabras con su parte de la oración:

```
import nltk
nltk.download('averaged_perceptron_tagger')
lotr_pos_tags = nltk.pos_tag(words_in_lotr_quote)
```

- ▶ **Resultado:**

Se obtiene una lista de tuplas, donde cada tupla contiene una palabra y su etiqueta gramatical correspondiente.

'It', PRP	'business', NN	'out', RP
's', VBZ	'Frodo', NNP	'your', PRP\$
'a', DT	'going', VBG	'door', NN
'dangerous', JJ		

- ▶ Esto prepara el texto para aplicar una gramática de **chunking**.

Gramática de Chunking

- ▶ **Chunking** requiere definir una **gramática**, que describe cómo se deben agrupar las palabras etiquetadas en frases (chunks).
- ▶ **Nota:**
Una **chunk grammar** es una combinación de reglas escritas generalmente con expresiones regulares (regex), que operan sobre secuencias de etiquetas gramaticales.

- ▶ **Ejemplo de regla para frases nominales (NP):**

grammar = "NP:{<DT>?<JJ>*<NN>}"

- ▶ **Interpretación:**

- ▶ NP significa *Noun Phrase* (frase nominal).
- ▶ La frase puede comenzar con un determinante opcional (<DT>?).
- ▶ Puede contener cero o más adjetivos (<JJ>*).
- ▶ Debe terminar con un sustantivo singular (<NN>).

- ▶ Esta regla identifica grupos como

“a dangerous business” o “your door”

como frases nominales.

Crear un Analizador de Chunking

- ▶ Una vez definida la gramática, **se crea un analizador de chunking usando** `RegexParser` de NLTK.
- ▶ Código para crear el analizador y aplicar la gramática:

```
# Crear el analizador de chunking  
chunk_parser = nltk.RegexpParser(grammar)
```

```
# Aplicar el analizador al texto etiquetado  
tree = chunk_parser.parse(lotr_pos_tags)
```

```
# Visualización del árbol de chunking  
tree.draw()
```

- ▶ Este código genera un árbol que visualiza las frases que se han agrupado según la gramática definida.

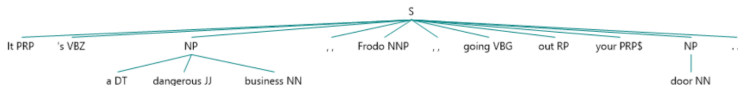
Guardar el Árbol de Chunking

- ▶ Es posible guardar la representación gráfica en un archivo.
- ▶ El siguiente código utiliza CanvasFrame y TreeWidget de NLTK para generar y guardar la figura:

```
from nltk.draw.util import CanvasFrame  
from nltk.draw import TreeWidget
```

```
cf = CanvasFrame()  
tc = TreeWidget(cf.canvas(), tree)  
cf.add_widget(tc, 10, 10) # coordenadas (x, y)  
cf.print_to_file("./fig/Notebook/tree.ps")  
                                # Guarda como archivo  
cf.destroy()
```

- La figura muestra el resultado del **chunking** aplicado a la cita.



Frases Nominales Detectadas

- ▶ A partir del árbol generado por el *chunk parser*, se identificaron dos frases nominales (*noun phrases*):
 - ▶ **“a dangerous business”**:
 - ▶ Contiene un *determiner* (“a”), un *adjective* (“dangerous”), y un *noun* (“business”).
 - ▶ **“door”**:
 - ▶ Es una frase nominal mínima que consiste únicamente en un *noun*.
- ▶ Estas frases se extrajeron utilizando la gramática:
$$\text{NP: } \{ \langle \text{DT} \rangle ? \langle \text{JJ} \rangle * \langle \text{NN} \rangle \}$$
que permite identificar construcciones comunes de frases nominales en inglés.

Ver código python.

Chinking: Exclusión de Patrones

El *chinking* es una técnica complementaria al *chunking* que permite **excluir patrones específicos** previamente incluidos dentro de una frase gramatical.

Mientras que el chunking se orienta a la *inclusión* de secuencias de palabras, el chinking se emplea para refinar dichas secuencias mediante la eliminación de elementos no deseados.

- ▶ Este proceso resulta especialmente útil cuando se requiere una selección más precisa de los componentes que conforman un *chunk*, eliminando aquellos tokens que cumplen con ciertos criterios gramaticales o estructurales.

Formalización

- ▶ Sea $c = \{(t_1, y_1), \dots, (t_k, y_k)\}$ un chunk previamente definido.
- ▶ El chinking aplica una operación de exclusión, notada por $\setminus$, tal que:

$$c' = c \setminus \{(t_i, y_i) \mid y_i \in \mathcal{E}\}$$

donde $\mathcal{E}$ representa el conjunto de etiquetas gramaticales a eliminar.

Ejemplo conceptual

- ▶ Sea un sintagma nominal inicialmente identificado como:

[The/DT very/RB big/JJ house/NN]_{NP}

Mediante chinking, se puede excluir el adverbio:

[The big house]_{NP}

- ▶ **Observación:**

La combinación de *chunking* y *chinking* permite construir reglas lingüísticas más expresivas y precisas para la identificación de frases en texto.

Ejemplo de uso

- Supóngase que se tiene el siguiente conjunto de tuplas, en el cual cada elemento contiene una palabra junto con su respectiva etiqueta de parte de la oración:

```
lotr_pos_tags = [  
    ('It', 'PRP'),           (" 's", 'VBZ'),  
    ('a', 'DT'),            ('dangerous', 'JJ'),  
    ('business', 'NN'),     (' ', ' '),  
    ('Frodo', 'NNP'),       (' ', ' '),  
    ('going', 'VBG'),        ('out', 'RP'),  
    ('your', 'PRP$'),        ('door', 'NN'),  
    ('.', '.')  
]
```

Gramática para Chunking y Chinking

- ▶ Se debe crear una gramática para determinar qué patrones se deben incluir chunks y cuáles se deben excluir chunk.
- ▶ Dado que se utiliza más de una regla, es necesario emplear comillas triples para definir la gramática

```
grammar = """
---Chunk: -{<DT>?<NN.*>}-#- Captura -sustantivos , -con
-----o -sin -determinantes , -sin -incluir -adjetivos
---Chink: -{<JJ>}-#- Excluye -adjetivos -(JJ)
-----"""
```

- ▶ En esta gramática:
 - ▶ La regla **Chunk** identifica secuencias de un determinante opcional seguido de uno o más sustantivos (como NN, NNS, NNP, etc.).
 - ▶ La regla **Chink** excluye específicamente los adjetivos (JJ) del chunk.

Resultado Esperado

- ▶ Aplicando esta gramática a la oración, se obtiene un chunk que agrupa sólo el sustantivo `business`, excluyendo el adjetivo `dangerous`:

```
[('a', 'DT'), ('business', 'NN')]
```

- ▶ Este resultado demuestra cómo el uso de **chinking** permite eliminar elementos no deseados (como adjetivos) dentro de un patrón de chunking más general, obteniendo grupos más específicos y controlados.

Ver código python.

Reconocimiento de Entidades Nombradas (NER)

El NER (*Named Entity Recognition*) es una técnica del procesamiento de lenguaje natural cuyo objetivo es **identificar frases nominales que hacen referencia a entidades específicas del mundo real**, tales como personas, lugares, organizaciones, fechas o cantidades.

- ▶ Mediante NER, no solo se detectan las entidades presentes en un texto, sino que también se determina la **categoría** a la que pertenece cada entidad identificada.

Formalización

Sea una secuencia de tokens etiquetados:

$$\mathcal{T} = (t_1, t_2, \dots, t_n)$$

El NER define una función:

$$\mathcal{N}(\mathcal{T}) = \{(e_1, c_1), (e_2, c_2), \dots\}$$

donde:

- ▶ e_i representa una entidad nombrada y
- ▶ c_i su categoría semántica correspondiente.

Ejemplo conceptual

- ▶ Dada la oración:

“Barack Obama was born in Hawaii”

el sistema de NER identifica:

[Barack Obama]_{PERSON}, [Hawaii]_{LOCATION}

- ▶ **Observación**

El reconocimiento de entidades nombradas constituye un **componente clave en aplicaciones como extracción de información, sistemas de pregunta–respuesta y análisis de textos a gran escala.**

Tipos de Entidades Nombradas según NLTK

- ▶ El siguiente cuadro muestra los tipos (y ejemplos) de entidades nombradas reconocidas por el paquete `nltk`:

Tipo de Entidad (NE)	Ejemplos
ORGANIZATION	Georgia-Pacific Corp., WHO
PERSON	Eddy Bonte, President Obama
LOCATION	Murray River, Mount Everest
DATE	June, 2008-06-29
TIME	two fifty a.m., 1:30 p.m.
MONEY	175 million Canadian dollars, GBP 10.40
PERCENT	twenty pct, 18.75 %
FACILITY	Washington Monument, Stonehenge
GPE	South East Asia, Midlothian

- ▶ Cada entidad detectada se clasifica automáticamente en una de estas categorías, enriqueciendo el análisis del texto con información estructurada y semánticamente relevante.

Aplicación de NER

► Preparación del Entorno:

Antes de utilizar `ne_chunk()`, es necesario descargar los clasificadores necesarios:

```
>>> import nltk
>>> nltk.download("maxent_ne_chunker")
>>> nltk.download("words")
```

► Aplicación sobre un Texto de Ejemplo:

Se reutiliza la variable `lotr_pos_tags`, que contiene la secuencia de palabras junto con sus etiquetas gramaticales:

```
>>> tree = nltk.ne_chunk(lotr_pos_tags)
```

Reconocimiento de Entidades Nombradas (NER)

- ▶ Al imprimir el árbol generado con `print(tree)`, se obtiene una representación jerárquica que identifica las entidades reconocidas:



- ▶ En este ejemplo, la palabra Frodo ha sido reconocida como una entidad del tipo PERSON, encapsulada dentro de una subestructura del árbol.

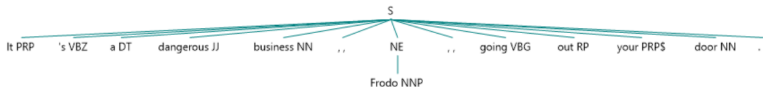
Reconocimiento Binario de Entidades Nombradas

- ▶ La función `ne_chunk()` de `nltk` permite una opción adicional mediante el parámetro `binary=True`.
- ▶ Esta configuración indica al analizador que detecte si una palabra o frase es una entidad nombrada, pero sin clasificar el tipo específico (como `PERSON`, `ORGANIZATION`, etc.).
- ▶ Ejemplo con `binary=True`

```
>>> tree = nltk.ne_chunk(lotr_pos_tags ,  
                           binary=True)  
  
>>> tree.draw()
```

Reconocimiento de Entidades Nombradas (NER)

- ▶ En este caso, la palabra Frodo ha sido identificada simplemente como una entidad nombrada, sin especificar si corresponde a una persona, organización o ubicación.



- ▶ Esta forma de análisis es **útil cuando se desea simplificar la salida del reconocimiento**, limitándose a identificar únicamente la presencia de entidades nombradas sin requerir una clasificación detallada.

Extracción Directa de Entidades Nombradas desde un Texto

- ▶ El reconocimiento de entidades nombradas también **puede aplicarse directamente a un fragmento de texto.**
- ▶ Para ello, se debe convertir el texto en una secuencia de palabras, etiquetarlas gramaticalmente, y posteriormente aplicar `ne_chunk()` con el parámetro `binary=True`.
- ▶ **Texto de Ejemplo:** *The War of the Worlds*

Men like Schiaparelli watched the red planet-it is odd, by-the-by, that for countless centuries Mars has been the star of war-but failed to interpret the fluctuating appearances of the markings they mapped so well. All that time the Martians must have been getting ready.

During the opposition of 1894 a great light was seen on the illuminated part of the disk, first at the Lick Observatory, then by Perrotin of Nice, and then by other observers. English readers heard of it first in the issue of Nature dated August 2."

- ▶ Se define una función `extract_ne()` que extrae todas las entidades nombradas del texto anterior, sin repetir ninguna.
- ▶ La función realiza las siguientes etapas:
 1. Tokenización del texto en palabras.
 2. Etiquetado gramatical (POS tagging).
 3. Análisis del árbol de entidades nombradas con `ne_chunk()`.
 4. Extracción y combinación de las entidades etiquetadas como "NE".

Extracción Directa de Entidades Nombradas desde un Texto

```
def extract_ne(quote):  
    words = word_tokenize(quote, language=language)  
    tags = nltk.pos_tag(words)  
    tree = nltk.ne_chunk(tags, binary=True)  
    return set(  
        "-".join(i[0] for i in t)  
        for t in tree  
        if hasattr(t, "label") and t.label() == "NE"  
    )
```

- ▶ Al utilizar `binary=True`, el resultado simplemente indica que se trata de entidades nombradas, sin distinguir si son personas, lugares u organizaciones.

Resultado

- ▶ Al ejecutar la función `extract_ne(quote)` sobre el texto de *The War of the Worlds*, el conjunto de entidades nombradas extraídas es el siguiente:

{'Lick Observatory', 'Mars', 'Nature', 'Perrotin', 'Schiaparelli'}

- ▶ Aunque **la ciudad de Nice no fue identificada correctamente**, probablemente debido a que NLTK la interpretó como un adjetivo regular en lugar de un lugar propio, se han extraído varias entidades nombradas con éxito.

- ▶ Estas son algunas de las categorías reconocidas:
 - ▶ **Institución:** Lick Observatory
 - ▶ **Planeta:** Mars
 - ▶ **Publicación:** Nature
 - ▶ **Personas:** Perrotin, Schiaparelli
- ▶ El resultado muestra una variedad significativa de entidades nombradas, lo que indica que el sistema es capaz de reconocer no solo personas y lugares, sino también instituciones y conceptos abstractos como planetas y publicaciones.

Ver código python.

Corpus Lingüístico

Un *corpus* es un **conjunto organizado de textos que se utiliza como base para el análisis y modelado del lenguaje natural.**

Estos textos **pueden provenir de diversas fuentes y dominios**, y suelen estar estructurados para facilitar su procesamiento computacional.

- ▶ Los corpus permiten estudiar patrones lingüísticos, entrenar modelos estadísticos y evaluar algoritmos de procesamiento de lenguaje natural en contextos reales y controlados.

Formalización

- Sea un corpus definido como:

$$\mathcal{C} = \{T_1, T_2, \dots, T_m\}$$

donde cada T_i representa un documento o texto individual.

- El análisis de un corpus se realiza a partir del estudio conjunto de sus documentos.

► Ejemplos de corpus:

Las bibliotecas de PLN incluyen colecciones textuales variadas, tales como:

- **Novelas y obras literarias** (por ejemplo, textos del Project Gutenberg).
- **Discursos históricos y políticos**, como discursos inaugurales de presidentes.
- **Textos periodísticos, conversaciones y documentos técnicos.**

► Herramientas.

La biblioteca **NLTK** proporciona acceso a múltiples corpus predefinidos, facilitando su exploración y uso en tareas de investigación y enseñanza del PLN.

Descargar y Cargar un Corpus

- ▶ NLTK proporciona varios corpus, que abarcan textos clásicos como *Moby Dick* de Herman Melville o colecciones específicas como los discursos inaugurales de los presidentes de los Estados Unidos.
- ▶ Para acceder a los textos, es necesario descargar el paquete de corpora de NLTK:

```
nltk.download("book")  
from nltk.book import *
```

Al cargar el corpus, se tiene acceso a varios textos:

- ▶ text1: *Moby Dick* de Herman Melville (1851)
- ▶ text2: *Sense and Sensibility* de Jane Austen (1811)
- ▶ text3: *El libro de Génesis*
- ▶ text4: Corpus de discursos inaugurales
- ▶ text5: Chat Corpus
- ▶ text6: *Monty Python and the Holy Grail*
- ▶ text7: Wall Street Journal
- ▶ text8: Personals Corpus
- ▶ text9: *The Man Who Was Thursday* de G. K. Chesterton (1908)

Exploración del Corpus de Anuncios Personales

- ▶ Un ejemplo interesante es el `Personals Corpus`, que contiene anuncios personales, los cuales eran una versión temprana de las citas en línea.
- ▶ Estos anuncios se publicaban en periódicos, y las personas interesadas podían responder para establecer contacto.
- ▶ Para explorar este corpus, simplemente se hace referencia al mismo de la siguiente manera:

```
>>> text8
```

- ▶ Con este corpus, se pueden realizar **análisis interesantes sobre cómo las personas se describían a sí mismas y qué características buscaban en otras personas**, lo que ofrece una ventana única a la naturaleza humana.

Concordance (Concordancia)

Una *concordancia* es una herramienta de análisis lingüístico que **permite visualizar todas las ocurrencias de una palabra dentro de un corpus**, junto con su **contexto inmediato**.

- ▶ Mediante el uso de concordancias, es posible observar cómo una palabra es utilizada a nivel de oración, así como identificar los términos que suelen aparecer en su entorno cercano.

Formalización

- ▶ Sea un corpus $\mathcal{C} = \{T_1, T_2, \dots, T_m\}$ y una palabra objetivo w .
- ▶ La concordancia define el conjunto:

$$\mathcal{K}(w) = \{\text{contexto}(w, T_i)\}_{i=1}^N$$

que contiene todas las apariciones de w junto con su contexto local.

Ejemplo conceptual

- ▶ Para una palabra objetivo w , la concordancia muestra instancias del tipo:

$$(\dots, w_{-k}, \dots, w_{-1}, \mathbf{w}, w_{+1}, \dots, w_{+k})$$

donde w_{-i} y w_{+i} representan las palabras que preceden y siguen a w dentro de una ventana de contexto.

- ▶ **Aplicaciones:**

- ▶ Análisis del uso semántico y sintáctico de palabras.
- ▶ Identificación de colocaciones y patrones léxicos.
- ▶ Apoyo en estudios lingüísticos y lexicográficos.

Uso de Concordancias

- ▶ En este ejemplo, se analiza la palabra "man" dentro del Personals Corpus (corpus de anuncios personales), disponible como text8:

```
>>> text8.concordance("man")
```

- ▶ Este comando devuelve todas las ocurrencias de la palabra "man" junto con su contexto.
- ▶ En este caso, se obtuvieron 14 coincidencias, algunas de las cuales son:
 - ▶ GENUINE ATTRACTIVE MAN 40 y . o ., no ties...
 - ▶ WELL DRESSED emotionally healthy man 37...
 - ▶ MARRIED MAN 42yo 6ft , fit , seeks Lady...

Búsqueda de un Hombre Honesto

- ▶ Entre las 14 coincidencias encontradas para la palabra "man", destacan tres instancias que reflejan un interés específico en hombres honestos, lo cual puede interpretarse como una pauta sociolingüística dentro de este tipo de anuncios personales:
 - ▶ SEEKING HONEST MAN
 - ▶ Seeks 35 - 45 , honest man with good SOH...
 - ▶ genuine , caring , honest and normal man...
- ▶ Estas expresiones muestran cómo se valora la honestidad como atributo fundamental en el contexto de relaciones personales.
- ▶ El uso de concordancias facilita este tipo de análisis cualitativo, al revelar patrones lingüísticos recurrentes y significativos.

Análisis de la Palabra “woman”

- ▶ Para investigar si existen patrones similares a los observados con la palabra "man", **se aplica el mismo procedimiento utilizando la palabra "woman"** en el Personals Corpus:

```
>>> text8.concordance("woman")
```

- ▶ Se encontraron 11 coincidencias, algunos ejemplos destacados son:
 - ▶ Seeking an honest , caring woman , slim or med . build...
 - ▶ like to meet full figured woman for relationship...
 - ▶ Seeking Christian Woman for fship , view to rship...
 - ▶ WOMAN OF SUBSTANCE 56 , 59 kg ., 50 , fit...

Comparación del Uso de “woman” y “man”

- ▶ Al analizar las concordancias encontradas con la palabra "woman", se observa que:
 - ▶ La honestidad como atributo explícito aparece en una sola ocasión:
"Seeking an honest, caring woman".
 - ▶ En contraste, la palabra "man" aparecía asociada a la honestidad en tres ocasiones distintas.
 - ▶ Otras cualidades mencionadas con frecuencia incluyen: apariencia física, orientación religiosa, y disposición para relaciones íntimas o de pareja.
- ▶ Este análisis revela que, si bien las concordancias no ofrecen una visión completa del corpus, permiten identificar patrones lingüísticos y temáticos que pueden ser significativos para estudios sociolingüísticos o de género.

Ver código python.

Dispersion Plot

Una *gráfica de dispersión léxica* (*dispersion plot*) es una herramienta de análisis que **permite visualizar cuántas veces** aparece una palabra en un corpus y **en qué posiciones** del texto ocurre.

- ▶ Este tipo de representación facilita el estudio de la distribución de términos a lo largo de un corpus, permitiendo comparar el uso relativo de distintas palabras y detectar patrones de aparición.

Formalización

- ▶ Sea un corpus representado como una secuencia ordenada de tokens:

$$\mathcal{C} = (t_1, t_2, \dots, t_n)$$

- ▶ Para una palabra objetivo w , la dispersión se define como el conjunto de posiciones:

$$\mathcal{D}(w) = \{ i \mid t_i = w \}$$

- ▶ La gráfica de dispersión representa visualmente el conjunto $\mathcal{D}(w)$ sobre el eje del corpus.

Ejemplo conceptual

- ▶ Dado un conjunto de palabras objetivo, como:

{man, woman, male, female}

la gráfica de dispersión muestra la posición de cada ocurrencia de estos términos a lo largo del corpus, permitiendo comparar su frecuencia y distribución espacial.

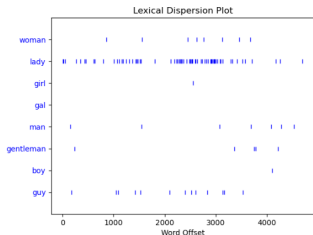
- ▶ **Aplicaciones:**
 - ▶ Comparación del uso de palabras y sus sinónimos.
 - ▶ Análisis de énfasis temático en distintas secciones de un texto.
 - ▶ Exploración preliminar de patrones lingüísticos en corpus extensos.

Gráfico de dispersión

- ▶ El siguiente código genera un gráfico de dispersión para las palabras “woman“, “lady“, “girl“, “gal“, “man“, “gentleman“, “boy“ y “guy“ dentro del Personals Corpus:

```
>>> text8.dispersion_plot(  
...     ["woman", "lady", "girl", "gal", "man", "  
...     )
```

- ▶ Este gráfico muestra la frecuencia de cada palabra a lo largo del corpus y permite visualizar cómo se distribuyen los términos en el texto.



Interpretación del Gráfico de Dispersión

- ▶ Cada línea vertical azul en el gráfico de dispersión representa una instancia de una palabra.
- ▶ Cada fila horizontal de líneas azules corresponde al corpus completo.
- ▶ A partir de este gráfico se pueden hacer las siguientes observaciones:
 - ▶ La palabra ‘‘lady’’ fue utilizada con mucha más frecuencia que ‘‘woman’’ o ‘‘girl’’.
 - ▶ No se observaron instancias de la palabra ‘‘gal’’.
 - ▶ Las palabras ‘‘man’’ y ‘‘guy’’ fueron usadas un número similar de veces, siendo más comunes que ‘‘gentleman’’ o ‘‘boy’’.

- ▶ Los gráficos de dispersión son **útiles cuando se desea observar la distribución y aparición de palabras dentro de un texto o corpus.**
- ▶ En el caso de un solo texto, esto puede ayudar a ver qué palabras aparecen cerca unas de otras.
- ▶ Si se analiza un corpus **organizado cronológicamente, el gráfico puede mostrar cómo ha cambiado el uso de ciertas palabras con el tiempo.**

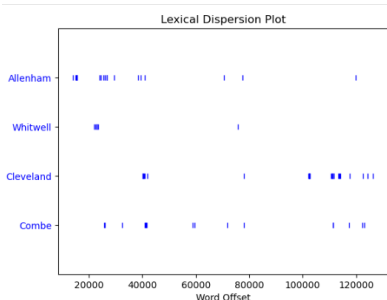
Dispersion Plot

- ▶ Se puede realizar un gráfico de dispersión para ver cómo y dónde se mencionan ciertas casas en la novela **Sense and Sensibility** (texto 2).

- ▶ En este caso, se analizan los nombres de varias casas:

```
>>> text2.dispersion_plot(["Allenham",  
                           "Whitwell", "Cleveland", "Combe"])
```

- ▶ Este gráfico muestra cómo se distribuyen los nombres de casas a lo largo del texto.



- ▶ A partir del gráfico de dispersión, se pueden hacer algunas observaciones sobre las casas **Allenham** y **Cleveland** en **Sense and Sensibility**:
 - ▶ **Allenham** se menciona mucho en el primer tercio de la novela, pero después su aparición disminuye considerablemente. Esto refleja el interés inicial de Marianne por Willoughby, ya que **Allenham** es la casa de la benefactora de Willoughby.
 - ▶ **Cleveland**, por otro lado, apenas aparece en los primeros dos tercios de la novela, pero muestra un aumento notable en el último tercio. Este cambio refleja la evolución de la relación entre Marianne y Willoughby, ya que **Cleveland** es la casa a la que Marianne viaja después de que su encuentro con Willoughby en Londres no sale como esperaba.

Frequency Distribution

Una **distribución de frecuencia** es una técnica de análisis que permite **identificar qué palabras aparecen con mayor frecuencia dentro de un texto o corpus**.

- ▶ Este análisis **proporciona una visión cuantitativa del vocabulario utilizado**, facilitando la detección de términos dominantes, patrones léxicos y posibles palabras clave del contenido analizado.

Formalización

- ▶ Sea V el vocabulario del corpus.
- ▶ La frecuencia de una palabra $w \in V$ se define como:

$$f(w) = \sum_{i=1}^n \mathbb{I}(t_i = w)$$

donde $\mathbb{I}(\cdot)$ es una función indicadora que vale 1 si la condición se cumple y 0 en caso contrario.

Ejemplo conceptual

- ▶ Dado un conjunto de tokens:

$$\mathcal{T} = \{t_1, t_2, \dots, t_n\}$$

la distribución de frecuencia contabiliza el número de apariciones de cada palabra distinta presente en el texto.

- ▶ **Aplicaciones:**

- ▶ Identificación de palabras más representativas de un texto.
- ▶ Análisis exploratorio previo al modelado estadístico.
- ▶ Construcción de representaciones como la bolsa de palabras.

Ejemplo

- ▶ Se utiliza la clase `FreqDist` de `nltk`, que es una subclase de `collections.Counter`:

```
from nltk import FreqDist
```

```
# Crear la distribución de frecuencia del corpus te  
frequency_distribution = FreqDist(text8)  
print(frequency_distribution)
```

- ▶ Este comando produce una salida como:

```
<FreqDist with 1108 samples and 4867 outcomes>
```

- ▶ Para analizar los resultados más relevantes, se extraen las 20 palabras más comunes:

```
frequency_distribution . most_common (20)
```

- ▶ Esta lista incluye puntuación y palabras vacías (stop words).
 - ▶ Algunas de las palabras más frecuentes: ', ', '. ', 'for', 'lady', 'seeks' y 'relationship'.
-
- ▶ Este análisis inicial revela patrones léxicos importantes, pero **puede afinarse aún más eliminando palabras stop words.**

Distribución de Frecuencia

- ▶ Para obtener una visión más significativa del léxico, se eliminan las *stop words*, que son palabras funcionales de poco contenido semántico.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
meaningful_words = [ word for word in text8 if
                     word.casefold() not in stop_words ]
```

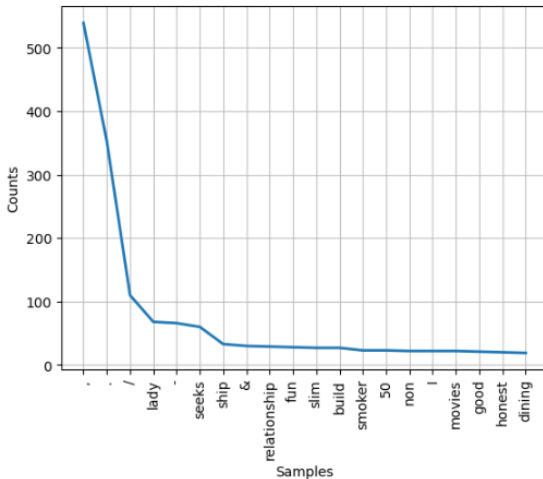
- ▶ Se construye una nueva distribución de frecuencia:

```
frequency_distribution = FreqDist(meaningful_words)
frequency_distribution.most_common(20)
```

- ▶ Algunas de las palabras más comunes: 'lady', 'seeks', 'relationship', 'fun', 'honest'.
- ▶ **Las palabras de puntuación aún dominan la frecuencia;** pueden ser eliminadas si se desea un análisis más limpio.

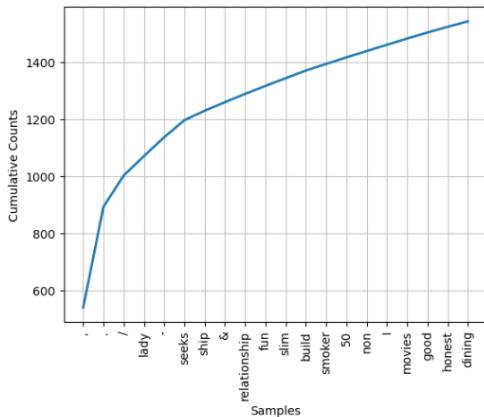
Visualización de la distribución

`frequency_distribution.plot(20)`



Visualización de la distribución acumulada

```
frequency_distribution.plot(20, cumulative=True)
```



Ver código python.

- ▶ Al observar las palabras más comunes tras eliminar las *stop words*, se identifican algunos patrones léxicos relevantes en los anuncios personales:
 - ▶ **Intereses personales:** 'fun', 'movies', 'dining', 'good'.
 - ▶ **Valores:** 'honest', 'non', 'smoker'.
 - ▶ **Características físicas:** 'slim', 'build'.
 - ▶ **Relaciones:** 'lady', 'seeks', 'relationship', 'ship'.
- ▶ **Patrón destacado:** las palabras 'slim' y 'build' aparecen con la misma frecuencia.
- ▶ Esto sugiere que **podrían estar siendo utilizadas en conjunto frecuentemente.**
- ▶ Ya se habían observado en contextos similares con el análisis de concordancia, **lo cual motiva la exploración de colocaciones.**

Collocations (Colocaciones)

Una *colocación* es una **secuencia de palabras que aparece con alta frecuencia en un texto o corpus, reflejando una asociación léxica significativa entre sus componentes.**

- ▶ Las colocaciones representan combinaciones de palabras que coocurren de manera recurrente y cuyo significado o uso no suele ser completamente deducible a partir de las palabras individuales por separado.

Formalización

- Sea un corpus representado como una secuencia de tokens:

$$\mathcal{C} = (t_1, t_2, \dots, t_n)$$

- Una colocación de longitud k se define como una subsecuencia:

$$(t_i, t_{i+1}, \dots, t_{i+k-1})$$

cuya frecuencia conjunta excede un umbral o presenta una alta medida de asociación estadística.

Ejemplo conceptual

- ▶ Algunos ejemplos comunes de colocaciones son:

“strong tea”, “make a decision”, “machine learning”

- ▶ Estas secuencias aparecen con mayor frecuencia de la esperada por azar.
- ▶ **Aplicaciones:**
 - ▶ Identificación de expresiones comunes del lenguaje.
 - ▶ Mejora de modelos de lenguaje y traducción automática.
 - ▶ Apoyo en tareas de extracción de información y análisis semántico.

Ejemplo en el Corpus: text8

- ▶ Colocaciones frecuentes en los anuncios personales (text8):

```
>>> text8.collocations()
```

- ▶ Salida (fragmento):
 - ▶ would like, medium build, slim build
 - ▶ social drinker, non smoker
 - ▶ long term, fun times, quiet nights
 - ▶ similar interests, never married
 - ▶ permanent relationship, well presented
- ▶ Estas colocaciones reflejan patrones de lenguaje frecuentes al describir preferencias, hábitos y estilo de vida en los anuncios personales.

Colocaciones Tras Lematización

- ▶ **¿Por qué lematizar antes de buscar colocaciones?**
 - ▶ Las colocaciones exactas pueden pasar desapercibidas si las palabras varían en forma.
 - ▶ Ejemplo: "likes" y "like" no se agruparán sin lematización.

Proceso:

```
lemmatized_words = [lemmatizer.lemmatize(word)
                     for word in text8]
new_text = nltk.Text(lemmatized_words)
new_text.collocations()
```

Resultado de las Colocaciones Tras Lematización:

- ▶ Al buscar colocaciones en el texto lematizado, **algunas combinaciones previas desaparecen**, como:
 - ▶ weekends away, poss rship
- ▶ **Nuevas colocaciones emergen**, como:
 - ▶ year old (muestra frecuencia de mención de edades).
 - ▶ photo pls (indica la solicitud frecuente de fotos).

Colocaciones encontradas:

medium build; social drinker; non smoker;
long term; would like; age **open**; easy going;
financially secure; Would like; quiet night;
Age **open**; well presented; never married;
single mum; permanent relationship; slim build;
year old; similar interest; fun time; Photo pls

Remarcas de colocaciones tras lematización

- ▶ Lematizar puede revelar combinaciones más generales.
- ▶ Algunas combinaciones de palabras previamente omitidas ahora son detectadas, ayudando a entender mejor el lenguaje de los usuarios.

Ver código python.

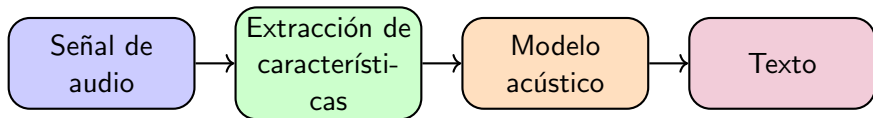
Biblioteca speech_recognition.

Es un módulo de Python que permite realizar **reconocimiento automático de voz (ASR)** a partir de señales de audio capturadas por micrófono o archivos digitales.

Características principales

- ▶ Captura de audio desde micrófono.
- ▶ Soporte para múltiples motores de reconocimiento:
 - ▶ Google Web Speech API
 - ▶ CMU Sphinx (offline)
 - ▶ Microsoft Azure
 - ▶ IBM Speech to Text
- ▶ **Conversión de señal de audio en texto.**
- ▶ Manejo de diferentes formatos: WAV, AIFF, FLAC.

Flujo básico de funcionamiento



Aplicaciones

- ▶ Asistentes virtuales
- ▶ Interfaces hombre-máquina
- ▶ Sistemas de transcripción automática
- ▶ **Control por voz en robótica**

Captura y Procesamiento de Audio

Inicialización del reconocedor

- ▶ Se crea una instancia del objeto `Recognizer`, que es responsable de procesar y reconocer el audio capturado.

```
recognizer = sr.Recognizer()
```

Nota: Se debe instalar:

- ▶ *SpeechRecognition*
- ▶ *pyaudio*

Captura de audio desde el micrófono

- ▶ Se abre **el micrófono del sistema como fuente de entrada de audio** dentro de un contexto controlado, garantizando que se cierre correctamente después de su uso.
- ▶ El método `adjust_for_ambient_noise` permite **calibrar el micrófono para filtrar el ruido ambiental**, mejorando la precisión del reconocimiento.
- ▶ **Se captura el audio emitido por el usuario** y se almacena en un objeto de tipo `AudioData`.

```
with sr.Microphone() as source:  
    print("Di algo...")  
    recognizer.adjust_for_ambient_noise(source)  
    audio = recognizer.listen(source)
```

Proceso de reconocimiento de voz

- ▶ El audio almacenado **se envía al servicio de reconocimiento de voz de Google** para su transcripción a texto.
- ▶ El parámetro `language='es-ES'` indica que el **idioma configurado** es español de España.
- ▶ Si la transcripción es exitosa, **el texto reconocido se imprime en pantalla**.
- ▶ Se captura la excepción que indica que el servicio no pudo interpretar el contenido del audio debido a ruido o habla ininteligible.
- ▶ Se maneja la excepción relacionada con fallos en la conexión al servicio de reconocimiento, como problemas de red o indisponibilidad del API.

Implementación en Python

```
# Intentar reconocer el audio
try:
    texto = recognizer.recognize_google(
        audio, language="es-ES"
    )
    print("Dijiste:", texto)

except sr.UnknownValueError:
    print("No-se-entendió-el-audio.")

except sr.RequestError as e:
    print("Error-al-conectarse-con-el-servicio:", e)
```

Ver código python.

Gracias por su atención.

